ADR 0001: Modular monolith with service and selector layers

Status	Accepted as the target design **inside each service**. Amended by ADR 0007: the vendor and admin side is a separate FastAPI service, so there are two modular monoliths, not one
Date	2026-10-19
Related	`03-system-architecture.md` section 4, the `shazlo-server` scaffold README

Context

Three engineers must ship an MVP in about two weeks and then keep growing the product. Today's backend is one Django app (`api`) where views hold business rules, the model layer imports the views layer, and modules cannot be changed independently. We need clear boundaries without the operating cost of several services.

Options

1. **Keep one `api` app.** Fastest today, but the coupling is the main reason fixes are risky.
2. **Microservices.** Strong isolation, but too much deployment, networking and monitoring work for three people.
3. **Modular monolith.** One deployable project split into domain modules with fixed layers and rules.

Decision

> **Amended (ADR 0007 and 0008).** The module list below describes the **Core** service, which now also owns `orders` (cart, checkout, stock, payments). The `vendors` module and the vendor half of `catalog` live in the **vendor-admin** service (`architecture.md`). The file roles and rules apply to both services (with FastAPI names: `routers.py`, `schemas.py`, `services.py`, `queries.py`).

Option 3. The structure and rules come from the `shazlo-server` scaffold:

- Modules: `users`, `auth`, `vendors`, `catalog`, `feed`, `orders`, `integrations`, plus a shared `core`, and an `api` package that is only the edge (GraphQL schema and REST routers).
- File roles in every module: controller (`graphql.py`, `apis.py`, `webhooks.py`), `services.py` (writes and rules), `selectors.py` (reads), `store.py` (Redis), `tasks.py` (thin, idempotent), `models.py`.
- A module may call another module's **services**, never its selectors, stores or models for writing.
- Expected errors use `ApplicationError`; anything else is a 500.
- Pure maths (feed mix, taste update, CSV validation) has no Django imports.
- Boundaries are enforced in CI with import-linter (to be added).

Consequences

- Easier testing and parallel work; a module can later be extracted behind its service interface.

- A little more ceremony per feature (a service function instead of logic in a view).
- The MVP does **not** restructure today's code; new code written for the MVP follows these names so migration is mechanical (`06-mvp-slice-and-migration.md` section 10).

Revisit when

A module needs independent scaling or a separate team owns it.